IOCP 구현 내용 설명

자료구조

- 게임 오브젝트를 관리하기 위해 GameObject* 배열을 사용하였습니다.
- 맵 지형 정보를 위해 char배열을 사용하였습니다.
- 이벤트 타이머를 위해 concurrency::concurrent_priority_queue를 사용하였습니다.
- 확장 OVERLAPPED구조체를 관리하기 위해 concurrency::concurrent_queue를 사용했습니다.
- 플레이어에게 보이는 플레이어를 관리하기 위해 std::unordered_set을 사용하였습니다.

구현 사항

- 7개의 스레드를 사용한 패킷, 로직 처리
- 로밍, 어그로, 고정 몬스터
- DB를 이용한 로그인 및 플레이어 정보 수정
- 타이머 큐를 이용한 이벤트 처리
- 시야 처리
- 패킷 조립

구현 사항 설명

7개의 쓰레드를 사용한 패킷, 로직 처리

- Lock사용을 줄이고 쓰레드간 의존성을 줄이기 위해 노력했습니다.
    1. 공유 메모리를 복사해 사용 후 다시 옮기기
    2. Non-Blocking 자료구조 사용

DataBase

- 저장 프로시저를 사용해 쿼리 파싱 오버헤드를 줄였습니다.

```
void DataBase::insert_player_data(const PlayerData& Player)
{
    if (!isConnect) {
        return ;
    }

    SQLHSTMT hstmt = 0;
    SQLRETURN retcode = SQLAllocHandle(SQL_HANDLE_STMT, hdbc, &hstmt);


    std::wstring command = std::format(L"EXEC insert_new_player '{}'", Player.name);

    retcode = SQLExecDirect(hstmt, (SQLWCHAR*)command.c_str(), command.length());
    if (retcode == SQL_SUCCESS || retcode == SQL_SUCCESS_WITH_INFO) {
    }
    else {

        HandleDiagnosticRecord(hstmt, SQL_HANDLE_STMT, retcode);
    }
    // Process data
    if (retcode == SQL_SUCCESS || retcode == SQL_SUCCESS_WITH_INFO) {
        SQLCancel(hstmt);
        SQLFreeHandle(SQL_HANDLE_STMT, hstmt);

    }
}
```

몬스터

- 로밍, 어그로, 고정 3종류의 몬스터를 구현했습니다.
- 로밍, 어그로 몬스터의 경우 A*알고리즘을 이용해 플레이어를 추적합니다.

```
int step = 0;
now->g = 0;
while (!Open.empty())
{
    auto pass = max_element(Open.cbegin(), Open.cend(), [&](Node* rhs, Node* lhs) {
        return rhs->f > rhs->f;
        });
    now = *pass;
    Open.erase(pass);
    step++;
    if (*now == end || step > MAX_STEP) { // 경로를 찾거나 너무 많은 시간을 쓰면 종료
        break;
    }
    Close.emplace_back(now->y, now->x);// 닫힌 경로에 추가하고

    int cur_y = now->y;
    int cur_x = now->x;

    //시작점에서 인접한 사각형 중 이동가능한 점을 확인
    for (int i = 0; i < 4; ++i) {
        int next_py = cur_y + dy[i];
        int next_px = cur_x + dx[i];
        if (next_px < 0 || next_py < 0) continue;
        if (next_px > MAX_LENGTH || next_py > MAX_LENGTH) continue;

        auto p = find(Close.begin(), Close.end(), Node(next_py, next_px));
        if (p != Close.end()) continue;

        Node* n = &nodes[MAX_LENGTH * next_py + next_px];
        if (n->g < now->g + 10) continue;

        n->x = next_px;
        n->y = next_py;

        n->g = now->g + 10;
        n->h = cal_h(end.x, end.y, n->x, n->y);
        n->f = n->g + n->h;
        n->pre = now;

        Open.push_back(n);
    }
}
```

타이머 큐

- Nonblocking 자료구조를 이용해 모든 스레드에서 이벤트를 추가 가능한 이벤트 타이머 큐를 사용하였습니다.
- 큐에서 이벤트를 빼 확인할 때 가장 짧은 이벤트 간격이 1초라는 것을 이용해 뺀 이벤트 중 1초보다 길게 남은 이벤트만 다시 이벤트 큐에 넣어 주었습니다.

```
while (true) {

    timer_event ev;
    while (!timer_queue.empty()) {

        while (!timer_queue.try_pop(ev));

        if (ev.start_time <= system_clock::now()) {
            //이벤트 시작
            do_timer_event_act(ev);
        }
        else {
            //기껏 뺐는데 다시 넣는거 좀 비효율 적이다.
            //다시 넣지 않는 방법으로 최적화 필요
            // 이벤트 중 가장 짧은 시간은 1초 만약 남은 시간이 1초 이상이면
            // 중간에 1초보다 짧은 이벤트가 끼어들 수 있다
            // 하지만 1초보다 짧다면?? 중간에 끼어들 이벤트가 생길 일이 없다
            // 즉 기다렸다가 해당 이벤트를 실행해도 된다는 것
            auto t = ev.start_time - system_clock::now();
            if (t < seconds(1)) {
                this_thread::sleep_for(t);
                do_timer_event_act(ev);
            }
            else
                timer_queue.push(ev);
            break;
        }
    }
    //큐가 비었거나 가장 짧은 이벤트가 1초기 때문에 1초 쉬어도됨
    this_thread::sleep_for(1000ms);
}
```

시야처리

- 플레이어가 움직였을 때 혹은 다른 플레이어가 움직였을 때 플레이어의 시야에 있는 플레이어 목록을 갱신해 시야에 있는 플레이어만 패킷을 보내도록 하였습니다.

- Lock이 걸려있는 시간을 최소화하기 위해 플레이어 목록 자료구조를 복사해 사용하였습니다.

```
//lock시간을 줄이기 위해 자료를 복사해서 사용
cl.vl.lock();
std::unordered_set<int> my_vl{ cl.viewlist };
cl.vl.unlock();


// 움직임으로써 시야에 들어온 플레이어 확인 및 추가
for (int other : nearlist) {
    // cl의 뷰리스트에 없으면
    if (0 == my_vl.count(other)) {
        // cl의 뷰리스트에 추가하고
        cl.vl.lock();
        cl.viewlist.insert(other);
        cl.vl.unlock();
        // 보였으니 그리라고 패킷을 보낸다.
        send_put_object(cl.index, other);

        //npc는 send를 안한다.
        //npc는 뷰리스트가 없고 자신을 볼 수 있는 플레이어가 있다면 isActive변수를 통해 움직인다.
        //플레이어에게 보인 NPC의 움직임 이벤트를 시작한다.

        //부하
        if (is_movable_monster(other)) {

            if (false == reinterpret_cast<movable_Npc*>(clients[other])->is_move)
            {
                reinterpret_cast<movable_Npc*>(clients[other])->is_move = true;
                timer_event t;
                t.ev = EVENT_NPC_MOVE;
                t.obj_id = other;
                t.start_time = std::chrono::system_clock::now() + std::chrono::seconds(1);
                timer_queue.push(t);
            }
            continue;
        }
        if (true == is_npc(other))continue;


        Client* otherPlayer = reinterpret_cast<Client*>(clients[other]);
        // 나한테 보이면 상대에게도 보인다는 뜻이니
        // 상대 뷰리스트도 확인한다.
        otherPlayer->vl.lock();

        // 상대 뷰리스트에 없으면
        if (0 == otherPlayer->viewlist.count(cl.index)) {
            // 뷰리스트에 추가하고 cl을 그리라고 전송
            otherPlayer->viewlist.insert(cl.index);
            otherPlayer->vl.unlock();
            send_put_object(other, cl.index);
        }
        // 상대 뷰리스트에 있으면 이동 패킷 전송
        else {
            otherPlayer->vl.unlock();
            send_move_object(other, cl.index);
        }

    }
```

```
    //계속 시야에 존재하는 플레이어 처리
    else {

        if (true == is_npc(other)) continue;
        Client* otherPlayer = reinterpret_cast<Client*>(clients[other]);
        otherPlayer->vl.lock();
        //상대방에 뷰리스트에 내가 있는지 확인
        if (0 != otherPlayer->viewlist.count(cl.index))
        {
            //있으면 이동
            otherPlayer->vl.unlock();

            send_move_object(other, cl.index);
        }
        else {
            //없으면 삽입
            otherPlayer->viewlist.insert(cl.index);
            otherPlayer->vl.unlock();

            send_put_object(other, cl.index);
        }
    }
}
```

```
// 움직임으로써 시야에서 빠진 플레이어 확인 및 제거
for (int other : my_vl) {
    // nearlist에 없으면
    if (0 == nearlist.count(other)) {
        // 나한테서 지우고
        cl.vl.lock();
        cl.viewlist.erase(other);
        cl.vl.unlock();
        send_remove_object(cl.index, other);

        //npc는 view리스트를 가지고 있지 않다.
        if (true == is_npc(other)) {
            continue;
        }
        Client* otherPlayer = reinterpret_cast<Client*>(clients[other]);

        // 상대방도 나를 지운다.
        otherPlayer->vl.lock();
        //있다면 지움
        if (0 != otherPlayer->viewlist.count(cl.index)) {

            otherPlayer->viewlist.erase(cl.index);
            otherPlayer->vl.unlock();

            send_remove_object(other, cl.index);
        }
        else otherPlayer->vl.unlock();
    }

}
```

패킷 조립

- TCP 프로토콜을 사용하였으므로 패킷이 잘려서 올 수는 있으나 순서와 에러 없음이 보장돼 있습니다. 그러므로 패킷이 중간에 잘린 경우, 여러 개의 패킷이 한 번에 도착한 경우만 고려해 패킷을 조립했습니다.

```
int remain_data = cl.prev_recv_size + num_byte;
unsigned char* packet_start = exp_over->_net_buf;
int packet_size = packet_start[0];

while (packet_size <= remain_data) {
    process_packet(client_id, packet_start);
    remain_data -= packet_size;
    packet_start += packet_size;
    if (remain_data > 0)
        packet_size = packet_start[0];
}


cl.prev_recv_size = remain_data;
if (remain_data) {
    memcpy_s(&exp_over->_net_buf, remain_data, packet_start, remain_data);
}

cl.do_recv();
```